

Milestone 3 & 4 — Todo List

Color pairing improvements & future enhancements for NILUT style transfer

MILESTONE 3: Smart Color Pairing

Replace blind percentile-based color pairing with nearest-neighbor matching + distance threshold. This is the core quality improvement for style transfer accuracy.

Current vs New Training Logic

Step	Current (Blind Percentile Sort)	New (Nearest-Neighbor Match)
1	Take 50K color samples from content	Take 50K color samples from content
2	Take 50K color samples from reference	Take 50K color samples from reference
3	Sort BOTH by percentile	For each content color, find NEAREST reference color
4	Pair them 1:1 by sorted position	If distance small → pair with reference color If distance large → pair with ITSELF
5	Train: input=content → output=ref	Train: input=content → output=paired_color

Key difference: Steps 3-4 change. Instead of blind percentile sorting, use nearest-neighbor matching with a distance threshold. Colors without a close match in the reference learn to stay as-is.

Why This Matters

Problem with Current	How New Fixes It
Sky blue forced to map to skin tone (wrong match)	Sky blue finds nearest blue in reference, or stays as-is
Every color MUST change, even if no good match exists	Colors without a close match keep their original value
Percentile position ≠ color similarity	Actual color distance (Euclidean in Lab space) used
Produces color artifacts in mismatched regions	Preserves original colors where style doesn't apply

Milestone 3 — Todo Checklist

3.1 Implement Nearest-Neighbor Pairing

- [] Replace histogram/percentile sort with KDTree nearest-neighbor lookup
- [] Use `scipy.spatial.KDTree` or `sklearn.neighbors.BallTree` for fast lookup
- [] Match in Lab color space (A,B channels) for perceptual accuracy

Pseudocode:

```
from scipy.spatial import KDTree

content_colors = sample_50k(content_images) # shape: (50000, 2)
ref_colors = sample_50k(ref_images) # shape: (50000, 2)

tree = KDTree(ref_colors)
distances, indices = tree.query(content_colors)

threshold = 15.0 # Lab units – tune this

paired = []
for i, (dist, idx) in enumerate(zip(distances, indices)):
    if dist < threshold:
        paired.append((content_colors[i], ref_colors[idx]))
    else:
        paired.append((content_colors[i], content_colors[i])) # keep original
```

3.2 Add Distance Threshold Parameter

- [] Add configurable threshold (default ~15 Lab units)
- [] Lower threshold = more conservative (fewer changes, more originals kept)
- [] Higher threshold = more aggressive (more colors remapped)

Threshold	Behavior	Use Case
5-10	Very conservative — only near-exact matches	Subtle color grading
10-20	Balanced — good matches only	General style transfer (recommended)
20-30	Aggressive — loose matches allowed	Strong style application
999+	Everything matches (same as current behavior)	Backwards compatibility

3.3 Update train.py

- [] Modify the color pairing function in `train.py`
- [] Add threshold as a CLI argument or config parameter
- [] Log stats: how many pairs matched vs kept-as-is (for debugging)

```
Example log output:
Style: moody_cinematic
Total pairs: 50,000
```

```
Matched (dist < 15): 32,450 (64.9%)  
Kept original (dist >= 15): 17,550 (35.1%)  
Avg match distance: 8.3 Lab units
```

3.4 Retrain & Validate

- [] Retrain all 5 per-style models + universal with new pairing
- [] Compare old vs new on validation images (side-by-side)
- [] Check: sky stays blue, skin stays natural, only style-relevant colors change
- [] Tune threshold if needed based on visual results

Success criteria: Style transfer applies style colors where appropriate, but preserves original colors in regions where the reference has no good match. No more forced color artifacts.

MILESTONE 4: Future Enhancements (If Time Permits)

These are stretch goals that build on top of Milestone 3. Only tackle these after M3 is solid.

4.1 Adaptive Threshold Per Style

- [] Instead of one fixed threshold, compute per-style threshold automatically
- [] Based on the color distribution overlap between content and reference
 - Styles with narrow color palettes (e.g., moody cinematic) get tighter thresholds
 - Styles with broad palettes (e.g., warm vintage) get wider thresholds

4.2 Weighted Pairing by Distance

- [] Instead of binary (match / keep original), use soft blending
- [] Close match = mostly reference color, far match = mostly original

```
weight = max(0, 1 - distance / threshold)
paired_color = weight * ref_color + (1 - weight) * content_color
```

- Produces smoother transitions instead of hard cutoff

4.3 Multi-Channel Matching (L + A + B)

- [] Currently matching on A,B channels only (chrominance)
- [] Optionally include L channel (luminance) for brightness-aware matching
 - Prevents dark shadows from being paired with bright highlights of same hue

4.4 Intensity/Strength Slider in Web UI

- [] Add a slider (0-100%) that controls how much style is applied
- [] Maps to the distance threshold internally
 - 0% = no change (threshold=0, everything keeps original)
 - 100% = full style (threshold=999, same as current behavior)
 - 50% = balanced (threshold=15, recommended default)

4.5 Batch Processing & Progress Feedback

- [] Process multiple images in one upload
- [] Show progress bar during style transfer
- [] Download all results as ZIP

Summary

Milest one	Focus	Priority	Status
M3	Smart color pairing (nearest-neighbor + threshold)	HIGH — Core quality fix	TODO
M4.1	Adaptive threshold per style	Medium	Stretch
M4.2	Weighted/soft pairing by distance	Medium	Stretch
M4.3	Multi-channel matching (L+A+B)	Low	Stretch
M4.4	Intensity slider in web UI	Medium	Stretch
M4.5	Batch processing + progress bar	Low	Stretch

Bottom line: Milestone 3 is the one change that matters most — replacing blind percentile sort with nearest-neighbor matching. Everything in M4 is a nice-to-have that builds on top of it.

Quick reference — The only code change in M3:

OLD: Sort both color arrays by percentile → pair 1:1 by position
NEW: `KDTree(ref_colors).query(content_colors)` → pair if distance < threshold, else keep original